```python
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
data = pd.read_csv('iris.csv.csv')

# Features and target
X = data.drop('species', axis=1)
y = data['species']

# Train-test split (80-20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# -------- Linear Kernel --------
svm_linear = SVC(kernel='linear')
svm_linear.fit(X_train, y_train)

y_pred_linear = svm_linear.predict(X_test)

print("Linear Kernel Accuracy:", accuracy_score(y_test, y_pred_linear))
print("Confusion Matrix (Linear):\n", confusion_matrix(y_test, y_pred_linear))


# -------- RBF Kernel --------
svm_rbf = SVC(kernel='rbf')
svm_rbf.fit(X_train, y_train)

y_pred_rbf = svm_rbf.predict(X_test)

print("\nRBF Kernel Accuracy:", accuracy_score(y_test, y_pred_rbf))
print("Confusion Matrix (RBF):\n", confusion_matrix(y_test, y_pred_rbf))
```

```
Linear Kernel Accuracy: 1.0
Confusion Matrix (Linear):
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

RBF Kernel Accuracy: 1.0
Confusion Matrix (RBF):
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

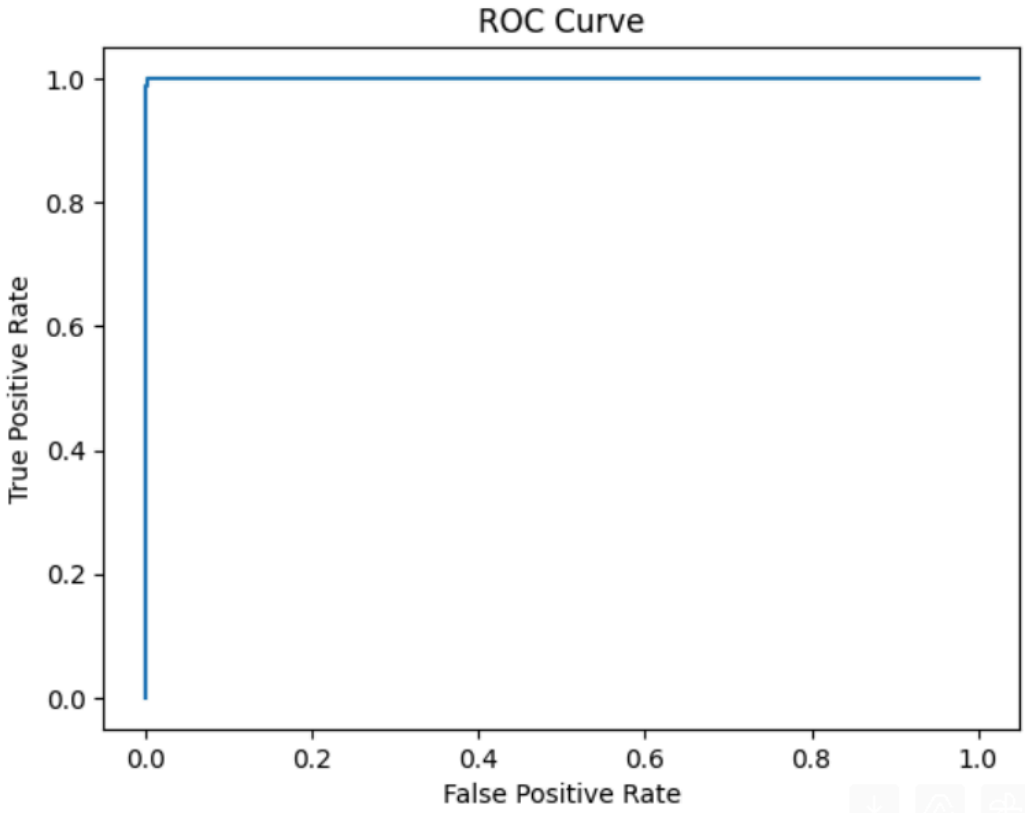
```python
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv('letter-recognition.csv')

# Features and target
X = data.iloc[:, 1:]   # Features
y = data.iloc[:, 0]    # Labels (letters)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train SVM
svm = SVC(kernel='rbf', probability=True)
svm.fit(X_train, y_train)

# Predictions
y_pred = svm.predict(X_test)

# Accuracy & Confusion Matrix
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```python
# -------- ROC & AUC --------
# Convert labels to binary format
lb = LabelBinarizer()
y_test_bin = lb.fit_transform(y_test)

# Predict probabilities
y_score = svm.predict_proba(X_test)

# Compute ROC and AUC for one class (example: first class)
fpr, tpr, _ = roc_curve(y_test_bin[:, 0], y_score[:, 0])
roc_auc = auc(fpr, tpr)

# Plot ROC curve
plt.plot(fpr, tpr)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.show()

print("AUC Score:", roc_auc)
```

```
Accuracy: 0.9305
Confusion Matrix:
 [[144   0   1   0   0   0   1   0   0   0   1   0   1   0   0   0   0   0
    0   0   1   0   0   0   0   0]
 [  0 143   0   5   0   1   0   0   0   0   0   0   0   0   0   0   0   4
    0   0   0   0   0   0   0   0]
 [  0   0 123   0   2   0   3   1   0   0   1   0   0   0   4   0   0   2
    0   0   1   0   0   0   0   0]
 [  0   1   0 153   0   0   0   2   0   0   0   0   0   0   0   0   0   0
    0   0   0   0   0   0   0   0]
 [  0   3   1   0 130   0   5   0   0   0   0   0   0   0   0   0   1   0
    0   0   0   0   0   0   0   1]
 [  0   2   0   0   1 134   0   0   1   0   0   0   0   0   0   0   0   0
    1   1   0   0   0   0   0   0]
 [  1   0   1   4   0   0 149   0   0   0   2   0   0   0   0   0   0   2
    0   0   0   1   0   0   0   0]
 [  0   4   0   8   0   0   0 106   0   0   5   0   0   1   2   1   1  13
    0   0   2   0   0   0   1   0]
 [  0   0   0   1   0   2   0   0 134   7   0   0   0   0   0   0   0   0
    0   0   0   0   0   2   0   0]
 [  0   0   0   1   1   1   0   0   3 139   0   0   0   0   1   0   0   0
    3   0   0   0   0   0   0   0]
 [  0   0   0   2   0   0   0   1   0   0 112   0   0   0   0   0   0  12
    0   0   0   0   0   3   0   0]
 [  0   0   1   0   4   0   3   0   0   0   0 142   0   0   1   0   1   1
    2   0   0   0   0   0   0   0]
 [  0   2   0   0   0   0   0   0   0   0   0   0 164   0   0   0   0   2
    0   0   0   0   0   0   0   0]
 [  1   0   0   3   0   0   0   0   0   0   0   0   0 139   5   0   0   2
    0   0   0   0   1   0   0   0]
 [  0   0   0   4   0   0   0   0   0   0   0   0   1   0 134   0   2   0
    0   0   1   0   3   0   0   0]
 [  0   1   0   0   0  14   8   0   0   0   0   0   0   0   0 148   0   0
    0   0   0   0   0   0   2   0]
 [  0   3   0   1   3   0   0   0   0   0   0   0   0   0   0   0 159   0
    0   0   0   0   0   0   0   0]
 [  0   5   0   2   0   0   0   0   0   0   2   1   0   2   0   0   0 148
    0   0   0   0   0   0   0   0]
```


```
[  0   3   0   1   3   0   0   0   0   0   0   0   0   0   0   0 159   0
   0   0   0   0   0   0   0   0]
[  0   5   0   2   0   0   0   0   0   0   2   1   0   2   0   0   0 148
   0   0   0   0   0   0   0   0]
```


```
[  0   2   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0   0
 167   0   0   0   0   0   0   1]
[  0   1   0   0   1   0   0   0   0   0   2   0   0   0   0   0   0   1
   0 153   1   0   0   2   1   1]
[  0   0   0   0   0   0   0   0   0   0   2   0   4   0   2   0   0   0
   0   0 170   2   3   0   0   0]
[  1   5   0   0   0   1   0   0   0   0   0   0   1   0   0   0   0   1
   0   0   0 147   2   0   0   0]
[  0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   0   3   0 145   0   0   0]
[  0   1   0   2   0   0   0   0   0   0   1   0   0   0   0   0   0   0
   0   0   0   0   0 150   0   0]
[  1   0   0   1   0   0   0   0   0   0   0   0   0   0   0   0   0   0
   0   1   0   1   0   0 164   0]
[  1   0   0   0   0   0   0   0   0   1   0   0   0   0   0   0   0   1
   4   0   0   0   0   0   0 125]]
```




```
AUC Score: 0.9999390030306781
```